ChatGPT

月面用分散処理ミドルウェア設計指針

Lu-MoS：ROS 2下位レイヤーで動作する月面分散実行基盤のMVP・研究設計

エグゼクティブサマリと実行プラン

未踏アドバンスト提案資料で定義されているLu-MoSの中心課題は、単なる「ROS 2ノードのオフロード」ではありません。提案書では、複数ローバーと共有計算サーバーを対象に、**通信状態、消費電力、温度、計算負荷、故障状態を観測しながら、実行場所・処理モデル・処理品質を動的に変更し、通信断や計算機故障時にもローバー側の最低限の安全機能を維持し、復旧後に状態を再同期する**ことが目標に置かれています。特に提案資料p.3では、切替1秒以内、計算ノード故障後の再開10秒以内、再同期30秒以内、スケジューラ判断10 ms以内など、すでにかなり強い定量目標が設定されています。filecitefileturn0file0

調査結果から、本プロジェクトで最も重要な設計判断は次の一点です。

> **「ROS 2の下にLu-MoSという巨大な独自通信ミドルウェアを新規開発する」のではなく、ROS 2が公式に設けているRMW境界を互換面として利用し、通信データプレーンにはZenoh、実行制御にはLu-MoS Agent、配置判断にはLu-MoS Scheduler、状態継続にはState Plane、ローカル安全にはSafety Planeを分離した“split-plane architecture”にする。**

ROS 2は、クライアントライブラリと通信ミドルウェアの間にRMWという抽象インターフェースを正式に設けています。RMWはDDS固有ではないよう設計されており、異なる通信ライブラリを背後に実装できる構造です。現在はZenohの公式RMWである`rmw_zenoh_cpp`もROS 2 Kilted以降フルサポートされています。したがって、**既存ROS 2ノードをほぼ変更せず新しい分散基盤を挿入するポイントとしてRMW境界を使うことには、ROS 2の設計思想上の正当性があります。** [1]

ただし、プロセスの起動・停止、コンテナ移動、チェックポイント、CPU/GPU割当、電力・温度監視まではRMWの責務ではありません。したがって、**「すべてをRMW内部でやる」という設計は推奨しません。** RMWはデータ経路の互換層、Lu-MoS AgentはOS・プロセス制御層、と責務を分離します。

さらに2026年7月にはNVIDIAなどから**ROSA**が発表され、ロボット群が共有サーバークラスGPUを利用し、ロボットのタスク要求と障害を考慮してスケジューリングする研究が出ています。公開結果では従来型の専用GPU配置に対して最大12.06倍の生産性改善を報告しています。これはLu-MoSの新規性に対する**非常に重要な新規競合**です。したがって、「ロボット群で共有GPUを使う」「タスクをサーバーへ逃がす」だけでは、2026年現在は未踏性の主張として弱くなっています。 [2]

Lu-MoSの研究仮説は、より狭く、より強く設定すべきです。

推奨するLu-MoSの技術的中心命題

> **通信途絶・電力・熱・故障・期限という月面特有の時変制約下で、mixed-criticalityなROS 2処理グラフについて、実行位置だけでなく処理品質・モデル・状態継続性まで同時に変更し、ネットワーク分断時には各ローバーが安全側に縮退し、再接続時には自動的に共有計算系へ復帰する分散実行基盤。**

これは「クラウドロボティクス」よりも、**Resilient Distributed Execution Substrate for Lunar Robotics**として位置づける方が強いです。

開発方針の要約

領域	MVPで採用	将来研究	理由
ROS互換面	`rmw_zenoh_cpp`	必要なら `rmw_lumos_cpp`	独自RMWを最初から作らない
データ通信	Zenoh	Zenoh + DTN gateway	ROS 2対応と柔軟なルーティング
実行制御	Rust製Lu-MoS Agent	Flight-grade Agent	プロセス・cgroup・コンテナ管理
配置判断	ルール＋制約付きcost function	MILP / learning-assisted	まず説明可能性を優先
状態移行	外部状態＋再起動型migration	CRIU等によるcheckpoint	「live migration」をMVPで約束しない
安全処理	ローバー固定	独立Safety Plane	通信断でも安全を維持
地球通信	MVPでは模擬	DTN/Bundle gateway	地球をリアルタイム制御ループに入れない
セキュリティ	mTLS + ACL	HW root of trust等	ZenohがTLS/mTLSとACLを提供[3]

週間実行プラン

期間	主タスク	完了条件
第1週	ROS 2 + `rmw_zenoh_cpp`、2 rover + server構成、計測基盤	ノード無改変で3計算機間通信、CPU/RTT/帯域を可視化
第2週	Rust Lu-MoS Agent、Workload Manifest、遠隔start/stop	`lumos run`から既存ROSノードを起動し、サーバー/ローバー間で再配置可能
第3週	Scheduler v0、通信・電力・熱条件を入力	条件変化で配置とモデル品質が自動変更
第4週	State Plane、lease/fencing、fault recovery	サーバーkill→安全縮退→別ノード再開→状態再同期
第5週	20台仮想ローバー、netem、セキュリティ、最適化版Scheduler	通信障害・温度・電力・故障を再現し比較実験可能
第6週	ベースライン比較、グラフ生成、DEMO演出、審査問答	提案書の主要定量目標について結果を示せる

2週間版では「通信状態に応じた自動offload/fallback」まで、**4週間版**では「故障＋state recovery」、**6週間版**では「20台スケール＋多目的スケジューリング＋比較評価」までを狙うのが現実的です。

要件定義

まず「月面だから○○msのネットワーク」と一つの数値を決めないことが重要です。JAXA自身、将来の月通信について、複数ユーザー・基地局・中継衛星が動的に移動し、中継切替、高遅延、高エラーレート、接続形態の変化を考慮する必要があるとしています。NASAも宇宙通信ではend-to-endリンクが利用不能になることを前提にDTNをstore-and-forward方式で運用しています。 [4]

また、地球―月間の光伝搬は平均距離で片道約1.3秒ですが、これは**地球―月バックホール**の物理下限であって、同じ月面上のローバーとローカル計算サーバーとの通信遅延ではありません。この二つを混ぜないことがシステム設計上非常に重要です。 [5]

前提条件と未指定事項

項目	現時点の扱い	設計上の意味
ローバー数	提案：実機相当2台、最大20台程度の負荷評価	2 physical + N virtualをMVP構成とする
共有サーバー台数	**未指定**	MVP 1台、設計上はN台対応
ローバーCPU/GPU	**未指定**	ISA/GPU固定のmigrationを前提にしない
月面ローカル無線方式	**未指定**	Ethernet/Wi-FiをMVPで模擬
月面ローカルRTT	**未指定**	単一値ではなくsweep評価
帯域	**未指定**	1 Mbps〜100 Mbps等を試験レンジとして使用。これは月面要求値ではない
Packet loss特性	**未指定**	Bernoulliだけでなくburst lossを評価
ローバー電力budget [W]	**未指定**	絶対Wではなく残量・power cap・J/taskで評価
計算機許容温度	**未指定**	CPU junction温度と月面外気温を混同しない
故障率/MTBF	**未指定**	確率を捏造せずfault injectionで感度分析
放射線SEU率	**未指定**	fault modelとして注入、実飛行値は機器選定後
safety control周期	**未指定**	safety処理は原則offload禁止
ROS 2 distro	**未指定**	MVPではZenoh RMWが正式対応したKilted系を候補
Linux/kernel	**未指定**	MVP Ubuntu/Linux、flight OSは別検討
セキュアブート/TPM	**未指定**	HW root-of-trustはMVP外
証明書有効期限/鍵更新方式	**未指定**	長期断絶環境を考慮したPKI設計が将来課題

月面の熱環境について、JAXAの有人与圧ローバー検討では南極域でおよそ高温側30°C、低温側−200°Cという外部環境が想定され、大きな昼夜温度差と限られた電力下での熱制御が課題とされています。ただし、**これはCPUの許容温度ではありません。** Lu-MoSが監視すべき値は外部温度そのものより、計算機内部の温度、thermal headroom、冷却能力、電力状態です。 6

宇宙放射線についても、NASA HPSCは放射線による電子部品の長期損傷と計算エラーを明示的な設計課題としており、ECC/EDAC、memory scrubbing、冗長ロジック、階層的なエラー検出・回復を採用しています。ミドルウェアだけで放射線問題を解決することはできませんが、**「計算ノードは時々嘘をつく・壊れる」ことをソフトウェア側のfault modelに含めるべき**です。 7

機能要件

ID	機能	必須設計
F-01	Resource Discovery	CPU/GPU/Memory/accelerator、利用可能モデルを発見
F-02	Health Monitoring	CPU負荷、温度、電力、battery、RTT、loss、bandwidth、heartbeat
F-03	Dynamic Placement	workloadをrover/server/別roverへ配置
F-04	Quality Adaptation	high/medium/lowモデル、FPS、resolution等を切替
F-05	Failover	process/node/network障害から代替実行
F-06	Partition Handling	通信断時にlocal safe profileへ自律遷移
F-07	State Continuity	map/mission/model stateのsnapshot、versioning、resync
F-08	ROS Transparency	ROSノードのソースコード変更を原則不要にする
F-09	Mixed Criticality	safety-critical処理をmigration対象から除外可能
F-10	Shared Resource Arbitration	複数ローバー間でCPU/GPUを優先度付き共有
F-11	Security	node identity、暗号化、認可、audit
F-12	Earth Gateway	地球系との通信をlocal realtime planeから分離
F-13	Explainability	「なぜ移動したか」をDecision Logとして記録
F-14	Fault Injection	network/node/power/thermal faultsを実験で再現

提案資料p.5も、ノード状態管理、スケジューラ、実行場所切替・状態移行、ROS 2/Zenoh接続、ロボット情報共有、安全機能、APIというほぼ同じ機能分解を示しており、この設計はその方針をより実装可能な責務境界へ落とし込んだものです。filecite turn0file0

非機能要件と評価値

以下は、**「提案資料で既に約束している値」**と**「MVPで追加推奨する評価値」**を分けるべきです。

指標	目標	種別
ローバー側計算消費電力	全処理ローカル方式の50%以下	提案目標

指標	目標	種別
同一電力での処理数	比較方式から30%以上増加	提案目標
20ローバー＋帯域制限時 deadline miss	≤5%	提案目標
地球向け通信量	raw dataの1/10以下	提案目標
通信断→ローカルfallback	≤1 s	提案目標
通信復旧→共有状態再同期	≤30 s	提案目標
計算ノード停止→代替実行	≤10 s	提案目標
thermal制約下の処理品質	最大品質の80%以上	提案目標
scheduler decision time	≤10 ms	提案目標 filecite turn0file0
scheduler decision P99	≤10 msを推奨	MVP評価方法
safety-local deadline miss	DEMO fault injection中0回	MVP実証目標。飛行保証値ではない
migration-induced stall	p50/p95/p99で計測	MVP追加指標
energy efficiency	J/taskまたはtasks/J	MVP追加指標
placement oscillation	migrations/minで計測	MVP追加指標
state freshness	Age of Informationで計測	MVP追加指標
recovery success	100回程度のfault injection成功率	MVP追加指標
mission availability	**未指定**	飛行要件として後日決定

ここで重要なのは、**平均値だけを評価しないこと**です。ロボットシステムでは平均レイテンシ10 msでも、P99.9が1秒であれば安全上問題になります。DEMOでは最低でもP50/P95/P99、deadline miss率、最悪復旧時間を併記すべきです。

既存技術調査と差別化

ROS 2そのものは計算配置システムではありません。ROS 2のRMWはdiscovery、serialization、transport等を抽象化し、DDS系またはZenoh系ミドルウェアに接続します。DDSはリアルタイムpublish/subscribeの標準で、ROS 2ではFast DDSやCyclone DDSなどが利用できます。一方Zenohも現在は正式なROS 2 RMWとして利用できます。 [8]

FogROS2はROS 2 launchを拡張し、どのコンポーネントをedge/cloudに配置するか指定するクラウドロボティクス基盤です。公開論文ではSLAM latency 50%削減、grasp planning 14秒→1.2秒、motion planning 28倍などを示していますが、基本的にはlaunch時のクラウド配置を容易にする思想です。 [9]

KubeEdgeはKubernetesをedgeまで拡張し、コンテナ配置、device management、cloud-edge metadata synchronization等を提供します。Sednaはその上でjoint inference、incremental learning、federated learningなどのedge-cloud協調AIを扱います。したがって非常に参考になりますが、ROS 2 graph semanticsや月面のmixed-criticality安全縮退は標準の問題設定ではありません。 [10]

主要技術比較

「成熟度」は本レポートにおける工学的評価であり、**高＝実運用OSS/標準として広く利用可能、中＝実用OSSだが今回用途への適合には拡張が必要、研究＝論文・研究システム中心**としています。

技術	主目的 / 層	ROSアプリ改変	強み	Lu-MoS用途での不足	License	主実装	成熟度
ROS 2 + RMW	robotics API / middleware abstraction	不要	middleware差替え点が公式に存在	process配置・thermal/power schedulerは持たない	coreはApache系、RMWごとに異なる	C/C++他	高 [1]
DDS / Fast DDS / Cyclone DDS	realtime pub/sub	不要	QoS、deadline、reliability等	compute placementではない	vendor依存	C/C++中心	高 [11]
`rmw_zenoh_cpp`	ROS↔Zenoh	不要	ROS 2 Kiltedからfull support、router/gossip型	workload lifecycleは管理しない	EPL-2.0系	C++/Zenoh-C	高〜中 [12]
Zenoh	data routing / pub-sub/query/store	API利用時は必要、RMWなら不要	heterogeneous edge向け、Rust基盤、router/P2P	process migration/schedulerなし	EPL-2.0 / Apache-2.0	Rust中心	高〜中 [13]
FogROS2	cloud robotics	launch変更	ROS 2 applicationをcloudへ容易に展開	月面断続・熱・電力・状態復旧が中心ではない	Apache-2.0	Python/ROS launch中心	中 [14]
KubeEdge	cloud-edge container orchestration	container化等必要	Kubernetes、edge agent、device twin	ROS semantics、短時間failover、安全制御には重い	Apache-2.0	Go中心	高 [15]
Sedna	edge-cloud AI	integration必要	joint inference、学習/推論協調	robot graph全体・状態migration・月面fault modelではない	Apache-2.0	Go/Python系	中 [16]
ROSA	robot fleet + shared GPU serving	abstraction利用	shared GPU pool、multi-model、task SLO、failure handling	公開概要では月面partition/thermal/DTN/mixed-critical safetyが主題ではない	論文実装ライセンス：**未指定**	Ray Serve/vLLM/PyTorch/JAX	研究 [2]

技術	主目的 / 層	ROSアプリ改変	強み	Lu-MoS用途での不足	License	主実装	成熟度
Ray	distributed execution	Ray API化必要	task/actor/resource scheduling	ROS transparency、lunar safety semanticsなし	Apache-2.0	Python/C++	高 [17]
CRIU	process checkpoint/restore	原則アプリ無改変	Linux process stateを保存・別環境でrestore可能	device/GPU/network/ISA依存を個別検証する必要	GPLv2	C	高〜中 [18]
DTN	disruption-tolerant networking	gateway integration	store-and-forward、断絶を前提	realtime local compute placementではない	標準/実装依存	複数	高 [19]
Lu-MoS案	lunar resilient execution	**ソース変更原則不要**	placement + quality + thermal/power/link/fault + state + safety	今後実証が必要	自プロジェクト決定	Rust推奨	研究

Zenohについては、ROS 2の`rmw_zenoh_cpp`がROS entityをZenoh session、publisher/subscriber/query等へマッピングし、ROS graph cacheも持っています。このためLu-MoSが独自に「ROS 2メッセージをZenohへ変換する層」を一から実装する合理性は低いです。まず公式実装を利用する方がMVP成功率が大幅に上がります。 [20]

競争上、どこを新規性にしてはいけないか

主張	2026年時点の強さ	理由
「ROSをクラウドへoffloadする」	弱い	FogROS2が既に存在
「ロボット群で共有GPUを使う」	弱い	ROSAが直接競合
「コンテナをedge/cloudへ配置する」	非常に弱い	Kubernetes/KubeEdge領域
「通信をZenoh化する」	新規性ほぼなし	ROS公式RMWが存在
「process checkpointを取る」	単独では弱い	CRIU等が存在
「通信断に耐える通信」	単独では弱い	DTNが宇宙標準領域
「通信・電力・熱・故障・deadlineを同時制約として処理位置＋品質を動的に変更」	**有望**	複数研究領域をcross-layerで統合
「partition時にsafety-localへ縮退し、復旧後statefulに共有計算へ戻る」	**有望**	単純cloud offloadとの差が明確

主張	2026年時点の強さ	理由
「既存ROSノードを変更せず、論理endpointを維持した実行位置変更」	**有望**	RMW/agent/state planeの統合課題
「mixed-criticality ROS graph scheduling」	**有望**	safety処理とAI処理を同じplacement問題にしない
「月面分散処理benchmark/fault trace」	**強い成果になり得る**	middleware本体とは独立した研究資産

特に**ROSAとの差分を今後のすべての説明資料に追加するべきです。** ROSAは2026年7月公開であり、共有GPU、multi-model pipeline、per-task performance requirement、failure handling、SLOベースのschedulerまで踏み込んでいます。Lu-MoSが単純な「ロボット版serverless/GPU pool」に見えると差別化が難しくなります。 [2]

したがって、新規性の一文は次のようにするのがよいでしょう。

> **Lu-MoSは、通信品質・電力・熱・計算機故障が時変しネットワーク分断が通常状態となり得る月面において、ROS 2処理グラフの実行位置、処理品質、状態継続性、安全縮退を統合的に管理するmixed-criticality distributed execution middlewareである。**

これは現時点では**「研究仮説として有望」**という意味であり、世界初を保証するものではありません。後述する追加文献調査と比較実験で狭義の新規性を固める必要があります。

推奨アーキテクチャ

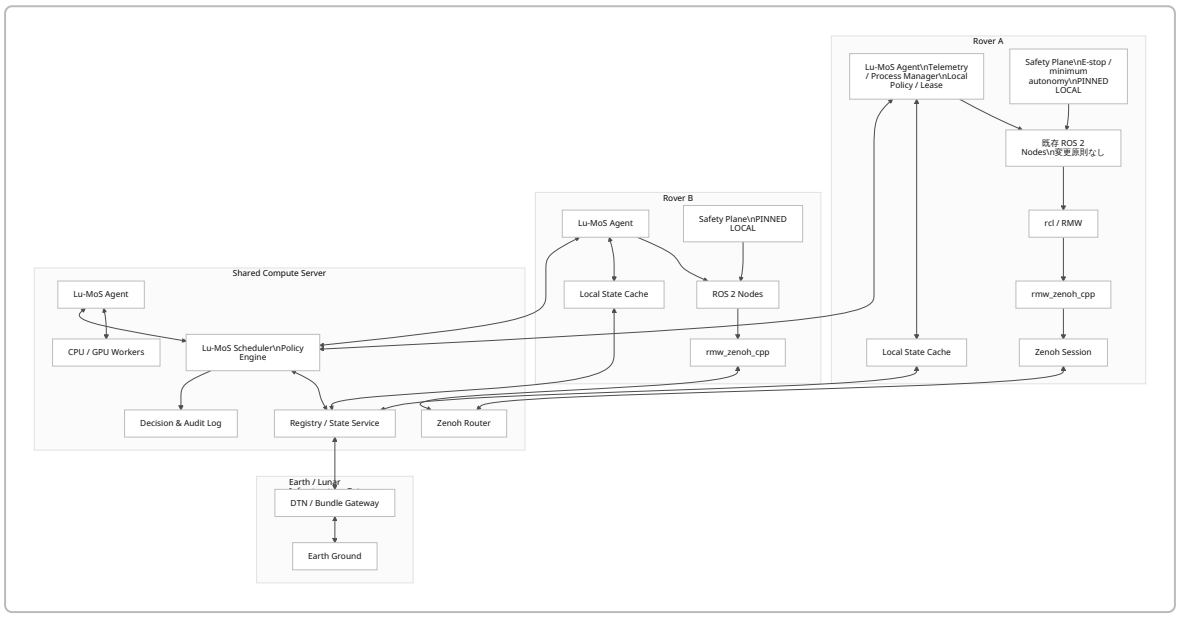
Lu-MoSを「ROS 2の下位レイヤー」と表現する場合、技術的には次の境界を推奨します。

```
ROS 2 application
       ↓
rclcpp / rclpy
       ↓
rcl
       ↓
============== ROS RMW boundary ==============
       ↓
rmw_zenoh_cpp        ← MVP
       ↓
Zenoh
       ↓
Network
```

RMWはROS 2が正式にmiddleware-specific codeを隔離するために設けた境界であり、`RMW_IMPLEMENTATION`によって実装を実行時選択できます。これは既存ROSアプリケーションを変更しないための最も自然なinterposition pointです。 [21]

しかし、Lu-MoSの本体はこの縦の通信スタックだけに入れません。横に**Lu-MoS Agent**を配置します。

コンポーネント図



この設計のポイントは、**Data Plane、Control Plane、State Plane、Safety Planeを分離すること**です。

Plane	責務	障害時の振舞い
Data Plane	ROS sensor/topic/service traffic	link品質に応じroute変更
Control Plane	placement、start/stop、quality変更	scheduler不在でもlocal policy継続
State Plane	map、mission state、model metadata、checkpoint	versioned cache + resync
Safety Plane	E-stop、最低限navigation/collision avoidance	**絶対にremote computeへ依存させない**
Earth Backhaul	telemetry、大容量データ、update	DTN/store-forward

NASAのDTNも、end-to-end connectionを要求せず各nodeでデータを保管して次hopが利用可能になった時にforwardする設計です。Lu-MoSではこの思想を**地球系のbackhaul**へ使い、ローカルロボットのmillisecond〜second級制御とは分離するべきです。 22

Lu-MoS Agent

各計算機に常駐するAgentをRustで実装します。提案書自体もRust/Zenohを想定しており、この方針と整合します。filecite turn0file0

Agentの責務は以下です。

Module	入力	出力
Telemetry Collector	`/proc`, cgroup, thermal, battery, network	ResourceSnapshot

Module	入力	出力
Network Monitor	RTT/loss/bandwidth/peer liveliness	LinkSnapshot
Process Manager	Scheduler command	start/stop/restart
Workload Runtime	manifest	process/container
State Manager	checkpoint/state hooks	StateVersion
Lease Manager	scheduler epoch	actuation authority
Local Policy	link/server failure	local fallback
Security Agent	cert/ACL identity	authenticated session
Audit	decision/events	trace/log

Agentの存在は重要です。RMWだけではOS processを別ホストで起動できないためです。

「既存ノードを変更しない」ためのAPI

MVPでは次のようなwrapper方式を推奨します。

```
lumos run \
 --workload perception-heavy \
 -- ros2 run my_perception detector_node
```

`lumos run`は内部で、

```
RMW_IMPLEMENTATION=rmw_zenoh_cpp
LUMOS_WORKLOAD_ID=perception-heavy
LUMOS_AGENT_SOCKET=/run/lumos/agent.sock
```

などを注入し、必要に応じてcgroup/containerへprocessを入れます。ROS 2のRMWは環境変数によって選択可能なので、この方式はROS application sourceに手を入れず実現できます。 23

ただし、**完全にゼロ設定ではありません。** workloadのdeadline、criticality、statefulness、必要GPU、fallback modelなどはROS graphから自動推測できないので、Manifestを要求します。

```
apiVersion: lumos/v1
kind: Workload
metadata:
 name: object-detection

execution:
 command: ["ros2", "run", "vision_pkg", "detector"]
 migratable: true
 criticality: mission
 deadline_ms: 100
```

```
resources:
  cpu_min: 1.0
  memory_mb: 512
  gpu:
    optional: true

state:
  mode: externalized
  checkpoint_period_ms: 1000

profiles:
  high:
    model: large
    target_fps: 20
  degraded:
    model: small
    target_fps: 5

fallback:
  location: rover
  profile: degraded

constraints:
  min_battery_percent: 20
  thermal_margin_c: 10
```

このManifestは**ROSアプリケーションのソース改変ではなくdeployment metadata**です。未踏DEMOではこの区別を明確に説明するとよいです。

状態移行は三段階に分ける

「ROSノードを別機体へlive migrationできます」と最初から主張するのは危険です。

CRIUはLinux processをfreezeし、memory/process stateをファイルへ保存し、後に別systemでrestoreできる仕組みを提供します。TCP socket stateにも対応機構があります。一方で、実際のROS nodeはmiddleware thread、network connection、device file、GPU runtimeなどを持つ可能性があるため、**任意ROS nodeの完全transparent migrationを保証できると仮定してはいけません。** [18]

したがって以下の段階設計が安全です。

Tier	方法	MVP	研究価値
Tier A	Stateless restart	◎	低
Tier B	Externalized logical state + restart	◎ **主方式**	高
Tier C	Process checkpoint/restore	△ 実験のみ	高
Tier D	accelerator-inclusive live migration	×	非常に高いが難しい

Tier BがLu-MoSに最も重要です。

たとえばSLAMを「Linux process memoryごと移す」のではなく、

```
Map state
Pose estimate
Mission progress
Model version
Last processed sequence
Calibration version
```

をLu-MoS State Planeへ外部化し、新しいprocessがそのlogical stateを読み込んで再開します。

つまり、研究の主張を

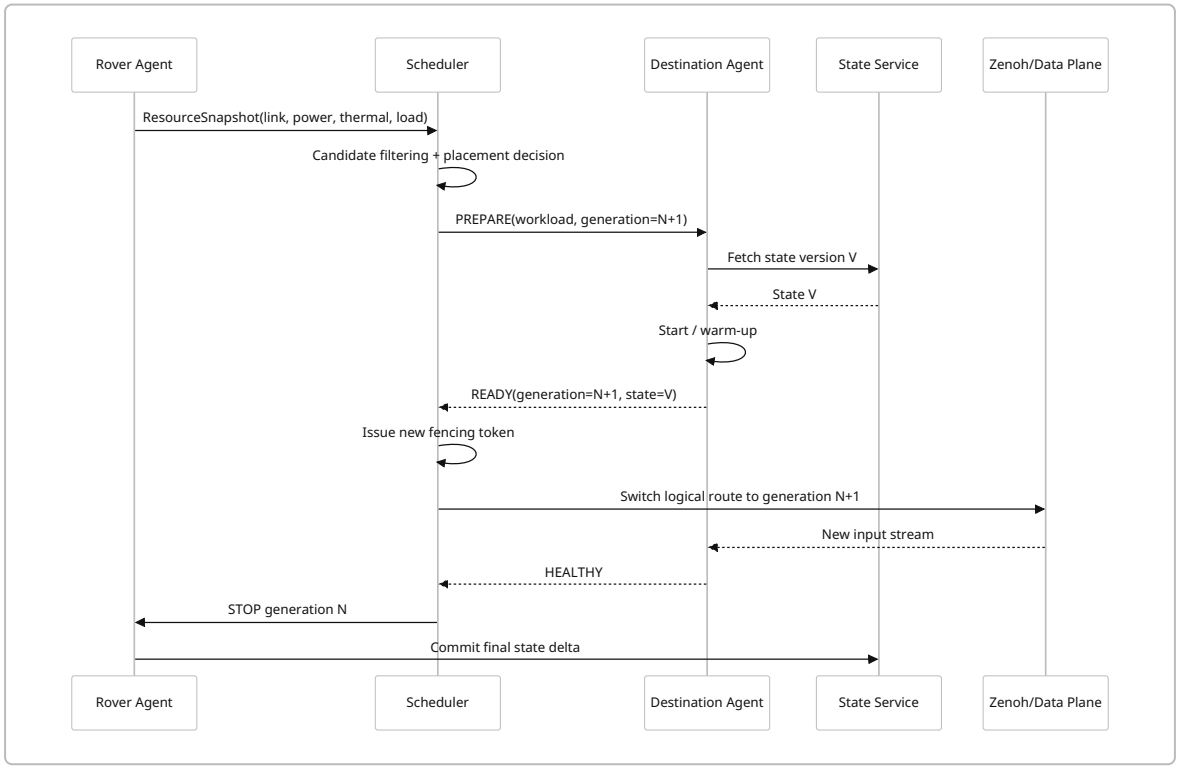
「process continuity」

ではなく、

「robotic task logical continuity」

へ置く方が強く、実装可能です。

Migration sequence



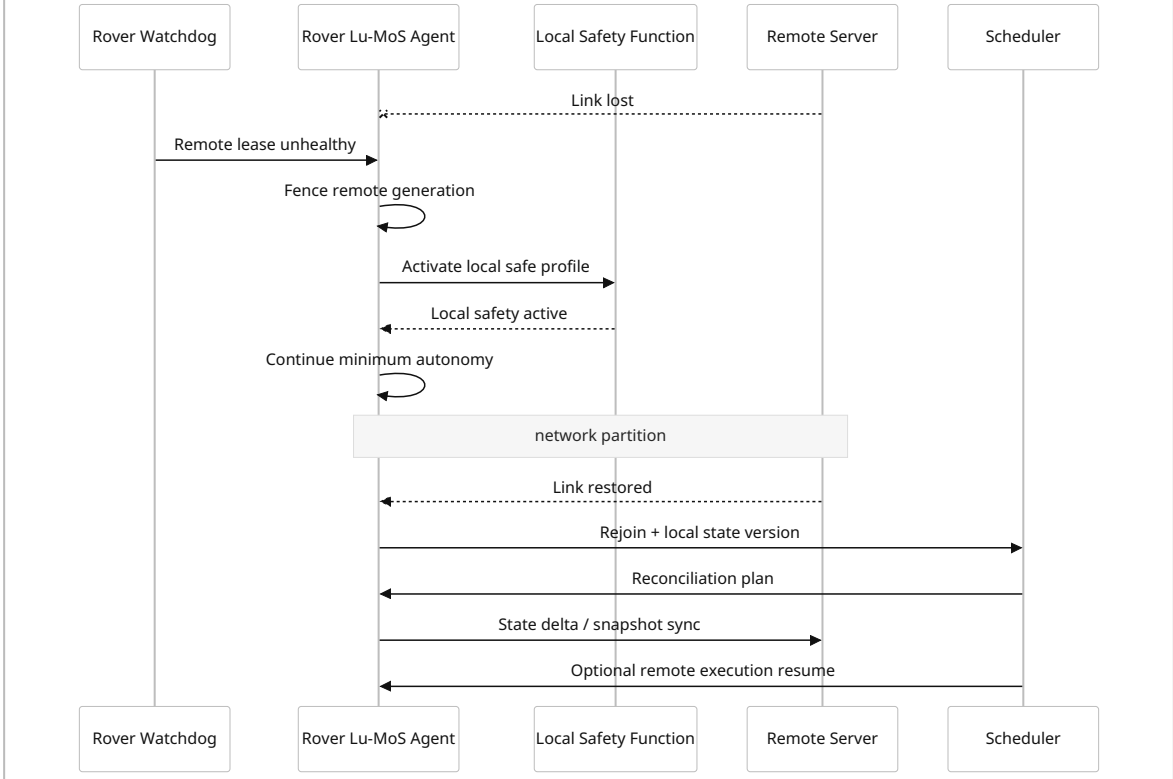
アクチュエータ出力を行う処理では、**old processとnew processが同時に命令を送るsplit-brainが最も危険**です。そのため各workloadに、

```
workload_id
generation
lease_id
lease_expiry
fencing_token
```

を持たせ、新しいgenerationだけがactuation authorityを持つようにします。

sensor processingのようなread-only workloadはmake-before-breakできますが、motor command publisherは**single-writer**にします。

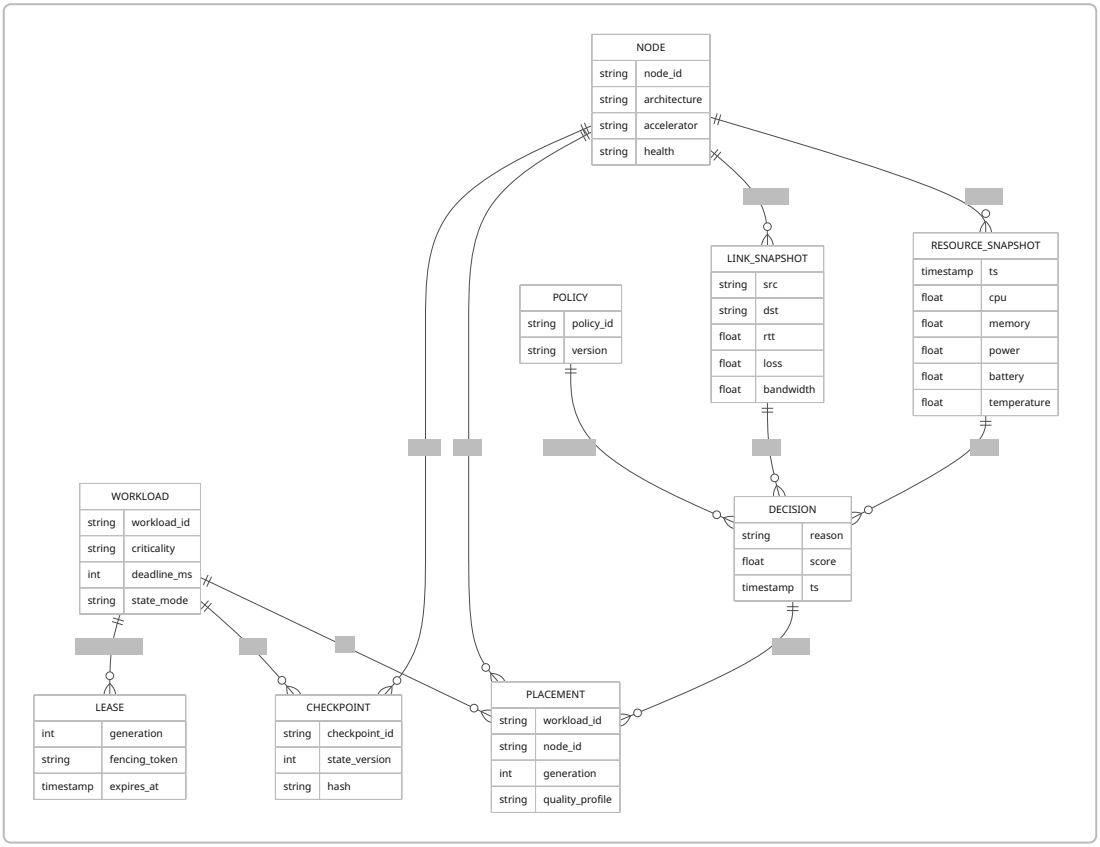
通信断時のsequence



ここでは**中央Schedulerの返事を待ってから安全動作へ切り替えてはいけません。** link lossそのものが中央Schedulerに到達できない理由になり得るからです。

NASAのsoftware fault-tolerance guidanceも、safe mode、独立監視、単純なbackup機能、off-nominal testを重視し、「rebootすれば必ず直る」という前提を置くべきでないとしています。 24

データモデル



このERモデルの利点は、審査員から「なぜ移動したのか」と聞かれた際、

```
Decision #8142
Reason:
  server-A thermal headroom < 5 °C
  rover-B battery = 18 %
  rover-A → server-B predicted latency = 61 ms < deadline 100 ms
  expected score improvement = 31 %
Action:
  move detector high → server-B medium
```

のように**decision provenance**を示せることです。学習系スケジューラへ進んでも、この説明可能性は残すべきです。

通信クラス

すべてのROS Topicを同じように扱うべきではありません。

Class	例	方針
Safety	emergency stop、最低限collision avoidance	rover-local固定

Class	例	方針
Real-time perception	camera→AI→obstacle	deadline/TTL付き、条件付きoffload
Shared state	map、pose、mission status	versioned/reliable、再同期可能
Bulk	log、raw imagery、checkpoint	bandwidth余裕時
Earth-bound	telemetry、science data	DTN/store-forward

地球―月通信についてはLunaNetもDTNをcore technologyとしており、通信断時にデータを保存して接続回復後に配送する思想です。[25]

セキュリティ

ZenohはTLSおよびmTLSをサポートし、key expressionやmessage operation、certificate identityなどに基づくACLも提供しています。ROS側の`rmw_zenoh`にもsecurity configuration用toolingがあります。[26]

MVPは次を最低ラインとします。

```
Node identity     : X.509
Transport         : mTLS
Authorization     : deny-by-default ACL
Control commands  : signed/authenticated + sequence/epoch
State integrity   : cryptographic hash
Checkpoint        : encrypted at rest
Software update   : signed artifact
Audit             : append-only decision/event log
```

ただしZenohの現行ACL設定はconfiguration変更にrestartを要するため、mission中に頻繁に動的ACLを書き換える設計にはしない方がよいです。事前にロールを準備し、動的なplacement authorityはLu-MoS lease層で処理する方が安全です。[27]

Secure boot、hardware root-of-trust、鍵の搭載方法、certificate renewal、失効リスト配送は**未指定**です。飛行システムでは別途Security Architectureとして設計する必要があります。

スケジューラと状態移行

Lu-MoSで最も研究価値が高い部分は、Zenohではなく**scheduler/policy engine**です。

単純に

```
CPU > 80% → serverへ移動
```

では未踏性は弱いです。

「処理位置」と「処理品質」を同時に決める問題として定式化します。

workload i、node j、quality/model profile qについて、

$$x_{ijq} \in \{0,1\}$$

を配置変数とします。

予測end-to-end latencyを、

$$L_{ijq} = C_{ijq} + N_{ijq} + Q_j + M_{ijq}$$

とします。

ここで、

- C：計算時間
- N：network transfer時間
- Q：queueing delay
- M：migration/warm-up cost

です。

energy cost:

$$E_{ijq} = E_{ijq}^{compute} + E_{ijq}^{network} + E_{ijq}^{migration}$$

thermal risk:

$$R_j^{thermal} = f(T_j, \dot{T}_j, T_j^{soft}, T_j^{hard})$$

reliability risk:

$$R_j^{fault} = f(heartbeat,\ ECC,\ reboot,\ recent\ faults,\ link\ stability)$$

を用意します。

重要なのは故障確率そのものが未指定なので、初期版で架空の「宇宙故障率」を入れないことです。 MVPではhealth scoreとして扱います。

目的関数の一例は、

$$\min \sum_{i,j,q} x_{ijq} \left[w_L \frac{L_{ijq}}{D_i} + w_E \frac{E_{ijq}}{E_i^{budget}} + w_T R_j^{thermal} + w_R R_j^{fault} + w_M M_{ijq} \right] + w_S \cdot SwitchingCost$$

です。

ただし、safetyやdeadlineは重み付きsoft objectiveだけにしてはいけません。

hard constraintsとして、

$$L_{ijq} \le D_i$$

$$T_j^{pred} < T_j^{hard}$$

$$Battery_j \geq Reserve_j$$

$$MemoryRequired_i \leq MemoryAvailable_j$$

$$Capability_i \subseteq Capability_j$$

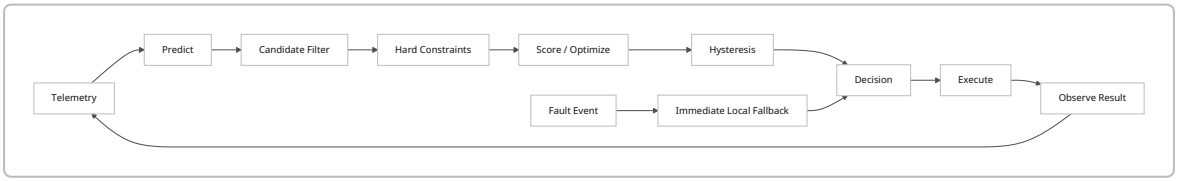
を入れます。

そしてSafety workloadについては、

$$x_{ijq} = 0 \quad \forall j \ / \ rover(\cancel{i})$$

として**placement問題から最初から除外**します。

Scheduler decision flow



単なる最低score選択では頻繁な行き来、すなわち**scheduler thrashing**が発生します。そのため、

```
new_score < old_score - Δ
```

を満たした場合のみ切り替えるhysteresisと、

```
minimum_dwell_time
```

を導入します。

ただしhard constraint violation時には待ちません。

推奨擬似コード

```
every scheduling_epoch:

    snapshot = collect_resource_and_link_state()

    # Safety workloads are never remotely scheduled.
    for task in safety_critical_tasks:
        enforce_local_placement(task)

    for task in migratable_tasks:

        candidates = []
```

```
        for node in available_nodes:

            if not compatible(task, node):
                continue

            prediction = predict(task, node, snapshot)

            if prediction.deadline > task.deadline:
                continue

            if prediction.temperature >= node.hard_thermal_limit:
                continue

            if prediction.battery_after < node.reserve:
                continue

            if prediction.link_health < task.minimum_link_health:
                continue

            score =
                w_latency     * normalize(prediction.latency) +
                w_energy      * normalize(prediction.energy) +
                w_thermal     * prediction.thermal_risk +
                w_reliability * prediction.failure_risk +
                w_migration   * prediction.migration_cost

            candidates.append(node, score)

        if candidates is empty:
            activate_degraded_local_profile(task)
            continue

        best = argmin(candidates.score)
        current = current_placement(task)

        if current violates hard_constraint:
            migrate(task, best, immediate=true)

        else if improvement(best, current) > hysteresis_delta
             and dwell_time_elapsed(task):

            prepare_destination(best)
            restore_logical_state(task, best)
            issue_new_generation_and_fencing_token(task)
            switch_route(task, best)
            retire_old_generation(task)

on remote_link_failure:

    fence_remote_actuation()
```

```
    activate_local_safe_profile_immediately()
    schedule_recovery_asynchronously()
```

比較すべきアルゴリズム

方式	MVP適性	強み	弱み	研究用途
Static placement	◎	baselineとして明快	適応不可	必須比較
CPU threshold rule	◎	実装容易、説明可能	多制約を扱いにくい	必須比較
Lexicographic rule	◎	safety/deadline優先が容易	global optimumでない	Lu-MoS v0推奨
Weighted cost	◎	実時間で軽い	weight調整必要	**Lu-MoS MVP推奨**
MILP	○	明確な制約付き最適化	台数増加で重い	offline oracle / v1
Heuristic search	○	大規模化しやすい	optimalityなし	v1
Contextual bandit	△	online adaptation	safe exploration困難	v2
RL	△〜×	複雑な長期最適化	data不足、OOD、説明性	研究段階
Learned predictor + hard scheduler	○	安全制約を維持しつつ予測改善	model validation必要	**将来最有力**

学習ベースを使うなら、「RLが最終決定する」構成より、

```
ML: latency / power / temperatureを予測
                  ↓
Deterministic constrained scheduler: 最終判断
```

を推奨します。

つまり**AIに安全制約を学習させない**。AIは予測モデルに限定し、最後はhard constraintsでshieldします。

ROSAも単純な単一request latencyではなく、robot fleet全体のSLOを満たすfactory productivityを最適化対象としており、Lu-MoSも「一処理を速くする」より「mission-level utility」を目的関数にする必要があります。Lu-MoSではさらにbattery/thermal/link partition/safetyをfirst-class constraintに入れることが差別化候補になります。[2]

状態整合性の原則

distributed systemとして、ネットワーク分断中に「全ローバーが常に完全一致するstate」を要求するのは適切ではありません。

Lu-MoSではstateを分類します。

State	consistency	例
Actuation authority	strong / single writer	motor control lease
Mission command	strongly versioned	current objective
Robot pose	latest-value / bounded staleness	localization
Shared map	eventual + conflict resolution	occupancy map
AI result	ephemeral	detection
Logs	eventual	diagnostics
Earth telemetry	DTN eventual	science/health

これにより、**「月面分散ロボットにどのconsistent modelを採用するか」**自体が研究テーマになります。

MVP・DEMO・評価

MVPで重要なのは「何かがサーバーへ移動しました」というDEMOではありません。

審査員が一目で見るべきなのは、

> **同じROS 2アプリケーションが、環境状態の変化に応じて“どこで・どの品質で”動くかが変化し、障害が起きても安全機能は止まらず、復旧後に自動で元の高性能状態へ戻る**

という一連の挙動です。

推奨MVPハードウェア

ミッション搭載hardwareは**未指定**なので、地上MVPと明示します。

役割	推奨MVP
Rover A	Raspberry Pi 5 / x86 mini PC
Rover B	Raspberry Pi 5 / x86 mini PC
Shared Compute	x86 workstation、GPUは任意
Network	Ethernet + Wi-Fi
Power measurement	USB-C power meter / smart power meter
Thermal	SoC sensor + optional external sensor
Fault emulator	Linux `tc/netem`
ROS	ROS 2 + `rmw_zenoh_cpp`
Middleware	Zenoh
Lu-MoS	Rust

役割	推奨MVP
Isolation	systemd/cgroup、必要ならPodman/Docker
Checkpoint experiment	CRIU
Simulation	ROS/GazeboまたはSynthetic publishers

Linuxのnetemはdelay、jitter、loss、duplicate、reordering等を設定でき、Gilbert-Elliott型のburst loss modelも扱えるため、通信障害の再現に適しています。 [28]

DEMO workload

Workload A：Perception

```
Camera → preprocessing → object detection → obstacle result
```

server側はhigh-quality model、rover側はsmall/degraded modelにします。

Workload B：Stateful Mapping

```
LiDAR/Depth → map update → map state
```

完全process checkpointではなく、map stateをState Planeへversionedに保存します。

Workload C：Safety

```
proximity sensor → emergency stop
```

これは絶対に移動しません。

この三つを同時に動かすことで、

```
AI = migratable
Map = stateful migratable
Safety = non-migratable
```

というmixed-criticality設計を視覚的に伝えられます。

最も強いDEMOシナリオ

画面には常に次を表示します。

```
┌──────────────────────────────────────────────────────────┐
│ Rover A      Rover B       Compute Server              │
│ CPU 34%      CPU 51%       GPU 71%                     │
```

```
| Pwr 8 W      Pwr 7 W       Temp 61°C                 |
| Battery 62%  Battery 81%   Link RTT 14 ms / loss 1% |
├──────────────────────────────────────────────────────┤
| workload            placement     quality           |
| detector-A          Server        HIGH              |
| detector-B          Server        HIGH              |
| mapping-A           Rover A       MEDIUM            |
| safety-A            Rover A       FIXED             |
├──────────────────────────────────────────────────────┤
| Last decision: SERVER_THERMAL_HEADROOM_LOW           |
| detector-A : Server → Rover A / HIGH → LOW           |
└──────────────────────────────────────────────────────┘
```

そして次の順番でfaultを注入します。

Phase	故障/制約	期待挙動	見せたい新規性
Normal	良好	heavy AIをserver	shared compute
Link degradation	bandwidth↓/loss↑	low modelをlocalへ	link-aware adaptation
Low battery	Rover A battery低下	computeをserverへ	energy-aware placement
Thermal	Server thermal headroom減少	workload分散/quality低下	thermal-aware scheduling
Server kill	process/node停止	≤10sで代替、safety継続	fault-aware recovery
Full partition	server unreachable	rover safety + minimum AI	autonomous degradation
Recovery	link復旧	state resync、high model復帰	stateful recovery
Scale	20 virtual rovers	load-aware arbitration	fleet scheduling

これを一本のDEMOとして流すことを強く推奨します。

「通信DEMO」「温度DEMO」「failover DEMO」を別々にすると既存技術の寄せ集めに見えます。

一つのmission executionの中で複数制約が重なることがLu-MoSの価値だからです。

ネットワーク試験条件

月面ローカルネットワークの統計値は**未指定**なので、「月面通信は100 ms」といった根拠のない固定設定を使わず、sweepします。

Parameter	DEMO sweep例	備考
RTT	1 / 10 / 50 / 100 / 300 ms	**試験レンジであり月面仕様値ではない**

Parameter	DEMO sweep例	備考
loss	0 / 1 / 5 / 10 / 20%	burst lossも追加
bandwidth	1 / 5 / 10 / 50 / 100 Mbps	試験レンジ
outage	0.5 / 1 / 5 / 30 / 60 s	partition recovery
server failure	process kill / host kill	2種類
thermal headroom	synthetic threshold sweep	actual lunar thermal modelではない
battery	80→20% synthetic	policy検証用

地球backhaulを追加DEMOする場合のみ、片道約1.3秒という物理伝搬遅延を踏まえ、2.6秒以上のRTT＋outageを模擬するのが妥当です。これをローカルrover-server通信には適用しません。 [5]

比較実験

最低限、以下4方式を同じ条件で比較します。

Baseline	内容
Local-only	すべてroverで処理
Static-offload	最初に決めたserver配置を固定
CPU-rule	CPU負荷だけで切替
Lu-MoS	link + energy + thermal + deadline + fault + quality

ここでFogROS2を無理に直接benchmark相手にする必要はありません。FogROS2はcloud deploymentを容易にすることが主眼であり、同じ問題設定ではないためです。むしろ**「最良のstatic placement」**をbaselineに含める方が、公平で強い実験になります。FogROS2についてはアーキテクチャ比較として扱います。FogROS2がlaunch時にcloud/edge componentを指定する構造であることは公式リポジトリにも明記されています。 [14]

計測するグラフ

最終発表では最低でも次を図にしてください。

図	x軸	y軸	示したいこと
Deadline miss	loss / rover数	miss %	degradation耐性
Energy efficiency	workload rate	tasks/J	shared compute効果
End-to-end latency	loss/bandwidth	P50/P95/P99	realtime性能
Recovery CDF	fault	recovery time	failover
Quality	power/thermal cap	accuracy/FPS	graceful degradation
Scheduler overhead	rover数	P99 decision time	10ms目標
Network traffic	mode	MB/min	offload overhead

図	x軸	y軸	示したいこと
Placement timeline	time	node/profile	**DEMOで最重要**

特に最後のtimelineは、

```
time
──────────────────────────────────────────────────────────────────────────────>

detector A
Server HIGH ████████████
            link loss
Rover LOW               ██████████
                      server recovered
Server HIGH                       ████████████████████

Safety A
Rover       ████████████████████████████████████████████████████████████████
```

という図にすると、Lu-MoSの概念が一枚で伝わります。

提案目標との評価traceability

提案書目標	DEMO方法	取得データ
power ≤50%	local-only vs Lu-MoS	Wh/J/task
throughput +30%	same power cap	completed tasks
miss ≤5% / 20 rover	20 virtual clients	deadline events
Earth data ≤1/10	raw vs semantic result	bytes
fallback ≤1s	forced partition	timestamp
resync ≤30s	restore link	version convergence
node failover ≤10s	kill server	first valid result
quality ≥80%	thermal/power cap	accuracy/FPS
decision ≤10ms	scheduler instrumentation	P50/P95/P99

これにより「できました」という主観DEMOから、**要件→試験→証拠のtraceability**を持つシステムエンジニアリング型DEMOになります。提案資料にすでにこれらの数値が置かれているため、非常に相性がよいです。turn0file0

想定審査員質問

質問	推奨回答
「FogROS2と何が違う？」	FogROS2はROS graphをcloudへ展開する基盤。Lu-MoSの研究対象は通信・電力・熱・故障・deadlineが同時に変わる中でのonline placement＋quality＋state recovery＋safety degradationです。FogROS2は重要な先行研究として比較します。 9
「Zenohが全部やってくれるのでは？」	Zenohは通信・routing・pub/sub/queryを担いますが、どのROS processをどこで動かすか、どのmodel qualityを選ぶか、battery/thermal/deadlineから判断するschedulerではありません。 13
「KubeEdgeでいいのでは？」	KubeEdgeは強力なedge container orchestrationです。ただしLu-MoSではROS graph semantics、sub-second safety fallback、robot state continuity、mixed-criticalityを第一級概念として扱います。KubeEdgeの設計は参考にします。 29
「ROSAと同じでは？」	これは最重要競合です。ROSAはrobot fleetのshared GPU servingとSLO schedulingを実現しています。Lu-MoSはRFM servingだけでなくROS graph全般を対象とし、通信partition、power、thermal、state resynchronization、safety-local partitionを統合する点を研究差分とします。ここは実験で証明します。 2
「本当にROSノードを変更しない？」	ソースコードは原則変更しません。RMWとlaunch wrapperを利用します。ただしdeadlineやstate policyを記述するdeployment manifestは必要です。特殊なstateful nodeではoptional state adapterが必要になる可能性があります。
「live migrationできる？」	MVPでは任意processのlive migrationは約束しません。外部化したlogical stateを新instanceへ復元するrestart migrationを主方式にします。CRIUは研究trackとして評価します。 30
「サーバーが壊れたら？」	serverを安全上のsingle point of failureにしません。safety/minimum autonomyは各rover固定です。共有server喪失は性能低下にはなっても直ちに安全喪失にはしません。
「ネットワークが完全に切れたらschedulerも使えないのでは？」	その通りです。したがってfallback判断は中央schedulerではなく各Lu-MoS Agentのlocal policyで実行します。
「なぜDDSではなくZenoh？」	DDSは強いrealtime/QoS基盤ですがcompute placementは担いません。ZenohはROS公式RMWが存在し、heterogeneous edge型構成に適するためMVPのdata planeに使います。ただしLu-MoSの価値はZenohそのものではありません。 31
「放射線をmiddlewareで解決できる？」	できません。hardware ECC/EDAC等と組み合わせる必要があります。Lu-MoSはnode health、corrupt state検知、failover、replica、checkpointというsoftware-level containmentを担当します。NASA HPSCも多層fault toleranceを採用しています。 7
「地球のクラウドを使えば？」	地球―月には光速だけでも片道約1.3秒があるため、低遅延安全処理を依存させません。地球はbulk processing/operations/data management側に置きます。 5

質問	推奨回答
「AI schedulerが暴走したら？」	AIはresource predictionに限定し、criticality、deadline、battery reserve、thermal hard limitなどはdeterministic hard constraintで保護します。
「月面実機がないのに月面向けと言える？」	現段階はflight qualificationではなくarchitecture/MVP validationです。通信、fault、power、thermalをfault injectionし、後続でHIL、thermal-vacuum、radiation/fault testへ段階的に進みます。JAXAも宇宙機には放射線、熱真空等の環境試験が必要としています。 [32]

リスク・研究課題・参考優先ソース

最大の研究リスクは、実装そのものではなく**「何をLu-MoS固有の発明として主張できるのか」**です。

2023年のFogROS2に加え、2026年にはROSAが共有GPU fleet schedulingへかなり踏み込んでおり、「ロボットの計算を共有サーバーで処理する」という説明だけでは既存技術との境界が薄くなっています。 [33]

そこで、未踏期間中に研究課題を次の形へ絞ることを推奨します。

研究として立てるべき問い

研究課題	新規性の可能性	実証方法
多制約joint placement	高	CPU-only/staticとの比較
placement + model quality joint optimization	高	throughput/energy/quality Pareto
network partition時のlocal autonomy	高	repeated outage injection
stateful task logical migration	高	mapping等でcontinuity測定
lease/fencingによる安全なendpoint migration	高	split-brain fault test
mixed-criticality ROS graph	高	safety + AI + stateful workload同時実行
thermal-aware scheduling	中〜高	temperature cap sweep
radiation/fault-aware orchestration	中〜高	bit/process/node fault injection
decentralized scheduler under partition	非常に高	scheduler分断実験
lunar distributed-compute benchmark	高	trace/data/tool公開

特に有望なのは、**中央schedulerが存在しないと動けないmiddlewareではなく、partition発生時に各ローバーがlocal policyへ移行し、再接続後にstateをreconcileする仕組み**です。

JAXAのLEV-2月面実証でも実際に通信の途切れやデータ欠損が発生し、搭載した異常検知・自己回復機能が動作したことが2026年に報告されています。これは「通信断は机上の極端ケースではなく、月面ロボットで現実に考慮すべきoff-nominal condition」であることを示す有力な日本語一次資料です。 [34]

主要リスク

リスク	重大度	対策
ROSAと新規性が重なる	高	lunar partition/power/thermal/state/mixed-criticality に焦点を絞る
独自RMW開発で時間消費	高	MVPは `rmw_zenoh_cpp` をそのまま利用
arbitrary process migrationが動かない	高	externalized logical stateを本線にする
GPU migrationが不安定	高	MVP要件から外す
Scheduler thrashing	中	hysteresis + dwell time
Shared server SPOF	高	local safety + replica設計
Network split-brain	非常に高	generation/lease/fencing
Power値がsimulationだけ	中	external power meterで最低1系統実測
Thermalがfakeに見える	中	synthetic inputとreal sensorを明確に区別
「月面」を名乗る根拠不足	高	JAXA/NASA constraint → test case traceabilityを作る
Securityが後付け	高	MVPからmTLS/identity導入
Fault rateを捏造してしまう	高	MTBF等は**未指定**のままsensitivity sweep
Safety certificationと誤解される	高	MVPとflight qualificationを明確に分離

NASAのsoftware guidanceでは、off-nominal testing、checkpoint/restart、independent monitoring、safe modeなどが推奨されており、Lu-MoSのfault injection中心の評価方針は宇宙ソフトウェアの一般的な設計思想とも整合します。[35]

未踏性を証明するために追加で調査する文献群

これからの文献調査では、「lunar middleware」だけで検索してはいけません。競合は異なる研究コミュニティに散在しています。

調査領域	検索すべき概念
Cloud Robotics	computation offloading, robot cloud, FogROS
Edge AI	collaborative inference, split computing, adaptive inference
Distributed Robotics	multi-robot resource sharing, distributed execution
ROS	dynamic node migration, RMW interception, ROS graph placement
Distributed Systems	stateful migration, fencing, leases, failover
Realtime	mixed-criticality scheduling, deadline-aware placement
Energy	energy-aware edge scheduling

調査領域	検索すべき概念
Thermal	thermal-aware workload scheduling
Space Networking	LunaNet, DTN, Bundle Protocol
Space Computing	fault-tolerant computing, HPSC
Robot Foundation Models	robot fleet inference serving, ROSA
Container migration	CRIU, checkpoint/restore
Fault tolerance	partition recovery, Byzantine/error containment

論文調査の最終成果物は単なる参考文献一覧ではなく、

```
                    placement quality state partition power thermal safety ROS-
transparent
FogROS2                 ○       △       ×       ×       ×       ×       ×       △
KubeEdge                ○       ×       △       ○       △       ×       ×       ×
Sedna                   ○       ○       △       △       ×       ×       ×       ×
ROSA                    ○       ○       ○?      fault   △       ?       △       ×
Lu-MoS target           ○       ○       ○       ○       ○       ○       ○       ○
```

という**claim matrix**にしてください。

「×が多いからLu-MoSが世界初」という意味ではありません。各セルに原著論文の根拠を付け、Lu-MoSの各claimを既存研究が本当にカバーしていないか検証するための表です。

最低限そろえるべき「未踏性の証拠セット」

審査までに以下の四つが揃えば、Lu-MoSはかなり説明しやすくなります。

第一の証拠は、Architecture Noveltyです。

```
ROS node unchanged
        |
        ▼
RMW/Zenoh data plane
        |
        ├──── Lu-MoS execution control
        ├──── State continuity
        ├──── Lunar resource scheduler
        └──── Local safety plane
```

既存技術の単なるforkではなく、責務境界を明確にします。

第二の証拠は、Behavioural Noveltyです。

一回のDEMO中に、

```
normal
→ network degradation
→ quality degradation
→ thermal constraint
→ server failure
→ local autonomous fallback
→ recovery
→ state resynchronization
```

までを連続して見せます。

第三の証拠は、Quantitative Noveltyです。

同じworkload、同じnetwork trace、同じpower capで、

```
Local-only
Static offload
CPU-only rule
Lu-MoS
```

を比較します。

第四の証拠は、Failure Semanticsです。

「速い」だけではなく、

```
When the system is partitioned, what exactly happens?
Who has control authority?
Which state is valid?
How is the old instance fenced?
When may the high-performance mode resume?
```

に仕様として答えます。

この4番目はFogROS2/Kubernetesの延長に見えにくく、**宇宙システムらしさと分散システム研究らしさを同時に出せる部分**です。

推奨する開発ロードマップ

```
Lu-MoS v0 — DEMO
 |
 ├── stock rmw_zenoh_cpp
 ├── Rust Agent
 ├── telemetry
 ├── rule/cost scheduler
 ├── restart migration
```

```
└── fault injection
        │
        ▼
Lu-MoS v0.5 — Research Prototype
 │
 ├── logical endpoint identity
 ├── state service
 ├── generation/lease/fencing
 ├── 20+ rover simulation
 ├── power/thermal predictive model
 └── optimization scheduler
        │
        ▼
Lu-MoS v1 — 独自研究成果
 │
 ├── decentralized partition-aware scheduling
 ├── state consistency semantics
 ├── optional rmw_lumos shim
 ├── checkpoint-assisted migration
 ├── DTN gateway
 └── lunar benchmark
        │
        ▼
Flight-oriented Lu-MoS
 ├── HIL
 ├── radiation/fault testing
 ├── thermal-vacuum
 ├── deterministic/safety analysis
 ├── HW root of trust
 └── flight computer integration
```

独自`rmw_lumos_cpp`は**v0では作らない**ことを強く推奨します。ROS 2自身のドキュメントも、新規RMW実装は相当量の実装・maintenance workになると説明しています。RMWの独自化が研究成果になると確認できてから投資すべきです。 [36]

参考優先ソース

今後の設計文書では、ブログより以下の一次資料を上位に置くことを推奨します。

優先度	資料	設計上の用途
最優先	**未踏アドバンスト Lu-MoS提案資料**	既存要求・定量目標・事業仮説のsource of truth □filecite□turn0file0□
最優先	ROS 2 Middleware Interface	独自middleware境界の設計根拠 [37]
最優先	ROS 2 Supported RMW / `rmw_zenoh`	Zenoh採用と既存ノード互換性 [38]
最優先	Eclipse Zenoh公式docs	data plane、TLS、ACL [39]
最優先	FogROS2原著/公式repo	cloud robotics比較 [40]

優先度	資料	設計上の用途
最優先・2026競合	NVIDIA ROSA原著	shared robot GPU/scheduler比較 [2]
高	KubeEdge公式architecture	edge orchestration比較 [29]
高	Sedna公式	collaborative inference比較 [41]
高	CRIU公式	checkpoint/migration feasibility [30]
最優先・宇宙	NASA DTN	partition/backhaul architecture [19]
最優先・宇宙	NASA LunaNet	lunar network architecture [25]
最優先・日本語	JAXA 月-地球間通信FS	月面通信の高遅延・高error・動的topologyの根拠 [42]
最優先・日本語	JAXA 有人与圧ローバー	thermal/power環境根拠 [43]
最優先・日本語	JAXA LEV-2月面実証	実際の通信途絶・自己回復の実績 [34]
高	NASA HPSC	radiation/power/fault-tolerant computing [7]
高	NASA Software Engineering Handbook	fault recovery、安全縮退、off-nominal testing [35]
高	OMG DDS / DDS Security	DDS比較、security reference [44]
実験	Linux kernel netem	communication fault injection [28]

最終的な設計判断としては、MVPの価値を「新しい通信プロトコル」に置かないことが最重要です。 ROS 2 RMW、Zenoh、CRIU、DTN、KubeEdge的なedge orchestrationなど、下位の構成技術には既に優れたものがあります。ROS 2自身もmiddleware差し替えを前提としており、Zenohは既に正式RMWに入っています。[1]

Lu-MoSが独自に作るべき核は、

Lunar Resource Model
＋ **Mixed-Criticality Workload Model**
＋ **Multi-Constraint Scheduler**
＋ **Safe Migration / Fencing Semantics**
＋ **State Reconciliation under Partition**
＋ **Quality-aware Graceful Degradation**

です。

そのため、最初のDEMO実装は **「ROS 2 Kilted系 + `rmw_zenoh_cpp` + Zenoh + Rust Lu-MoS Agent + rule/cost scheduler + externalized state + Linux netem」** という、かなり既存OSS寄りの構成で問題ありません。むしろ、既存OSSを通信やプロセス管理のために再発明せず、**“月面で処理をどこで・どの品質で・どの状態を保持しながら動かすべきか”という判断と故障時の意味論に開発時間を集中することが、未踏性とスタートアップとしての技術資産の両方を最大化する設計方針です。**

---

1 21 37 ROS 2 middleware interface
https://design.ros2.org/articles/ros_middleware_interface.html?utm_source=chatgpt.com

2 ROSA: A Robotics Foundation Model Serving System for Robot Factories | Research
https://research.nvidia.com/publication/2026-07_rosa-robotics-foundation-model-serving-system-robot-factories?utm_source=chatgpt.com

3 26 27 Access Control · Zenoh
https://zenoh.io/docs/manual/access-control/?utm_source=chatgpt.com

4 42 月-地球間通信システム開発・実証（FS） | JAXA宇宙戦略基金
https://fund.jaxa.jp/techlist/theme5/?utm_source=chatgpt.com

5 Voyager 1: What Is a Light-Day - NASA Science
https://science.nasa.gov/mission/voyager/voyager-1/voyager-1-what-is-a-light-day/?utm_source=chatgpt.com

6 43 有人与圧ローバー | JAXA 有人宇宙技術部門
https://humans-in-space.jaxa.jp/biz-lab/tech/pressurized-rover/?utm_source=chatgpt.com

7 High Performance Spaceflight Computing (HPSC) - NASA
https://www.nasa.gov/game-changing-development-projects/high-performance-spaceflight-computing-hpsc/?utm_source=chatgpt.com

8 31 36 38 Different ROS 2 middleware vendors — ROS 2 Documentation: Kilted documentation
https://docs.ros.org/en/ros2_documentation/kilted/Concepts/Intermediate/About-Different-Middleware-Vendors.html?utm_source=chatgpt.com

9 14 GitHub - BerkeleyAutomation/FogROS2: An Adaptive and Extensible Platform for Cloud and Fog Robotics Using ROS 2 · GitHub
https://github.com/berkeleyautomation/fogros2?utm_source=chatgpt.com

10 15 29 Why KubeEdge | KubeEdge
https://kubeedge.io/docs/?utm_source=chatgpt.com

11 44 About the Data Distribution Service Specification Version 1.4
https://www.omg.org/spec/DDS/?utm_source=chatgpt.com

12 GitHub - ros2/rmw_zenoh: RMW for ROS 2 using Zenoh as the middleware · GitHub
https://github.com/ros2/rmw_zenoh?utm_source=chatgpt.com

13 39 Docs · Zenoh
https://zenoh.io/docs/?utm_source=chatgpt.com

16 41 sedna/README.md at main · kubeedge/sedna · GitHub
https://github.com/kubeedge/sedna/blob/main/README.md?utm_source=chatgpt.com

17 ray-project · GitHub
https://github.com/ray-project?utm_source=chatgpt.com

18 30 GitHub - checkpoint-restore/criu: Checkpoint/Restore tool · GitHub
https://github.com/checkpoint-restore/criu?utm_source=chatgpt.com

19 22 Delay/Disruption Tolerant Networking - NASA
https://www.nasa.gov/communicating-with-missions/delay-disruption-tolerant-networking/?utm_source=chatgpt.com

20 rmw_zenoh/docs/design.md at rolling · ros2/rmw_zenoh · GitHub
https://github.com/ros2/rmw_zenoh/blob/rolling/docs/design.md?utm_source=chatgpt.com

23 rmw_implementation — rmw_implementation: Rolling 3.1.0 documentation
https://docs.ros.org/en/rolling/p/rmw_implementation/README.html?utm_source=chatgpt.com

24 35 7.23 - Software Fault Prevention and Tolerance - SW Engineering Handbook Ver D - Global Site
https://swehb.nasa.gov/spaces/SWEHBVD/pages/166592616/7.23%2B-%2BSoftware%2BFault%2BPrevention%2Band%2BTolerance?utm_source=chatgpt.com

25 LunaNet - NASA
https://www.nasa.gov/communicating-with-missions/lunanet/?utm_source=chatgpt.com

28 Family tc netlink specification — The Linux Kernel documentation
https://docs.kernel.org/next/networking/netlink_spec/tc.html?utm_source=chatgpt.com

32 宇宙機の環境試験の課題解決 | JAXA宇宙戦略基金
https://fund.jaxa.jp/techlist/theme2_24/?utm_source=chatgpt.com

33 40 FogROS2: An Adaptive Platform for Cloud and Fog Robotics Using ROS 2 - Michael Danielczuk
https://mjd3.github.io/publications/2023-fogros2?utm_source=chatgpt.com

34 JAXA | 変形型月面ロボット「LEV-2」による月面実証成果の国際学術誌「Science Robotics」論文採択
https://www.jaxa.jp/press/2026/06/20260618-1_j.html?utm_source=chatgpt.com